

# SAW HUB — Documentação Técnica

Arquitetura, modelo de dados, RBAC, segurança e deploy — guia de manutenção do sistema.

**01** Visão geral

---

**02** Arquitetura

---

**03** Modelo de dados

---

**04** Módulos & épicos

---

**05** Autenticação & RBAC

---

**06** Segurança

---

**07** Integrações externas

---

**08** Deploy & operação

---

**09** Como contribuir

---

# Como o SAW HUB é construído, por dentro.

Referência técnica para quem for dar manutenção, revisar ou estender o sistema — arquitetura de infraestrutura, modelo de dados completo, RBAC, segurança e o fluxo de deploy real usado em produção. Escrita a partir do código e da configuração efetivamente em uso, não de um plano à parte.

**Domínio** Plataforma SaaS de mentoria para restaurantes    **Hospedagem** Hostinger VPS · Coolify    **Última verificação** Fase 5 · Homologação, 2026

## 01 Visão geral

O SAW HUB centraliza a jornada do mentorado (agenda, metas, tarefas, atas, conteúdos, eventos e loja) e dá à SAW um painel de operação para o próprio negócio — comercial, financeiro e gestão de time. Dois perfis de acesso, uma base de código só.

### **Mentorado**

Dashboard, metas estratégicas, tarefas vinculadas a mentorias, histórico de mentorias e atas, biblioteca de conteúdos, eventos, loja e perfil com gamificação (XP/nível calculados em tempo de leitura, nunca persistidos).

### **Admin (SAW)**

Back-office da própria SAW: pipeline comercial e leads, DRE e fluxo de caixa, gestão de time com RBAC por área, curadoria de conteúdo/eventos/mentorias e um painel consolidado com ranking de todos os mentorados.

### **Diferencial de IA**

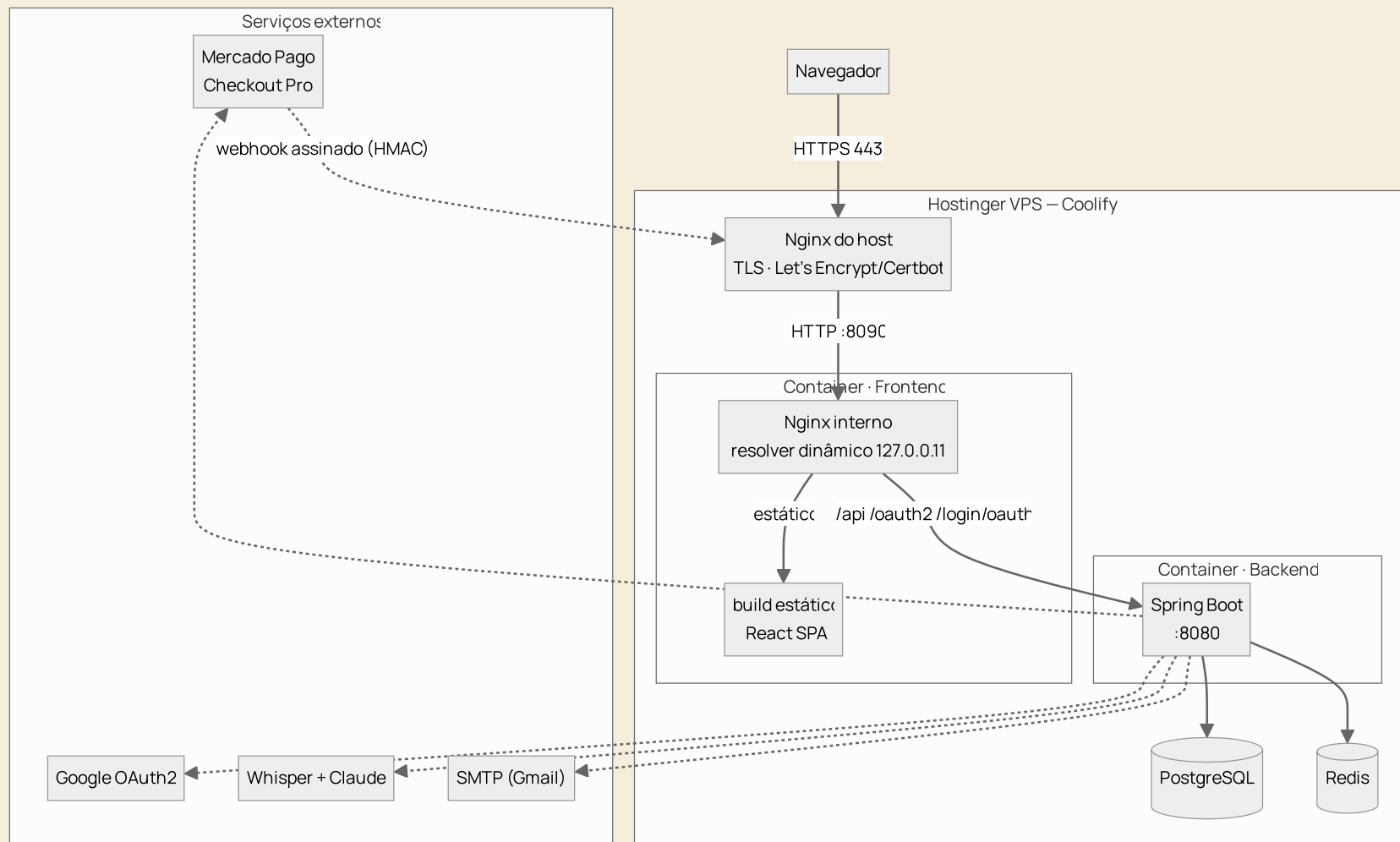
Upload do áudio da mentoria → transcrição (Whisper) → rascunho de ata (resumo + encaminhamentos sugeridos) via Claude — revisado por humano antes de publicar. Nunca processa ou decide nada sozinho.

| CAMADA      | TECNOLOGIA                                   | POR QUÊ                                                                                   |
|-------------|----------------------------------------------|-------------------------------------------------------------------------------------------|
| Frontend    | React + Vite + TypeScript                    | SPA sem SSR – app majoritariamente logado, sem necessidade de SEO.                        |
| Backend     | Java + Spring Boot (MVC, Data JPA/Hibernate) | Ecossistema mais maduro do mercado pra RBAC complexo (E15) e regras financeiras.          |
| Banco       | PostgreSQL + Redis                           | Relacional pra dados transacionais; Redis pra cache, sessão e rate limit.                 |
| Auth        | Spring Security + OAuth2                     | E-mail/senha + Google, sessão por cookie, CSRF, RBAC por área.                            |
| Agendamento | Spring @Scheduled                            | Casos de uso são jobs periódicos (e-mails, lembretes) – não fila de alto volume.          |
| Pagamento   | Mercado Pago (Checkout Pro)                  | Gateway externo – o SAW HUB nunca processa dado de cartão.                                |
| Deploy      | Hostinger VPS + Coolify                      | Front e back no mesmo servidor, deploy por git push, custo ~5x menor que PaaS gerenciado. |

- **Escala alvo do MVP:** 10–15 usuários, mas a arquitetura (Postgres + Spring Boot stateless) já nasce pensada pra crescer sem reescrita – não é otimização prematura, é não criar dívida óbvia.

## 02 Arquitetura

Um único servidor (VPS) hospeda front e back lado a lado, atrás do Coolify. O TLS é terminado no Nginx do host; daí em diante o tráfego trafega em HTTP puro dentro da rede interna do Docker até o Spring Boot.



Front e back são dois containers Docker separados na mesma rede do Coolify – o Nginx do container do frontend é quem decide, por path, o que serve como estático e o que repassa pro backend (mesma origem, sem CORS/SameSite=None em produção).

**Por que essa topologia**

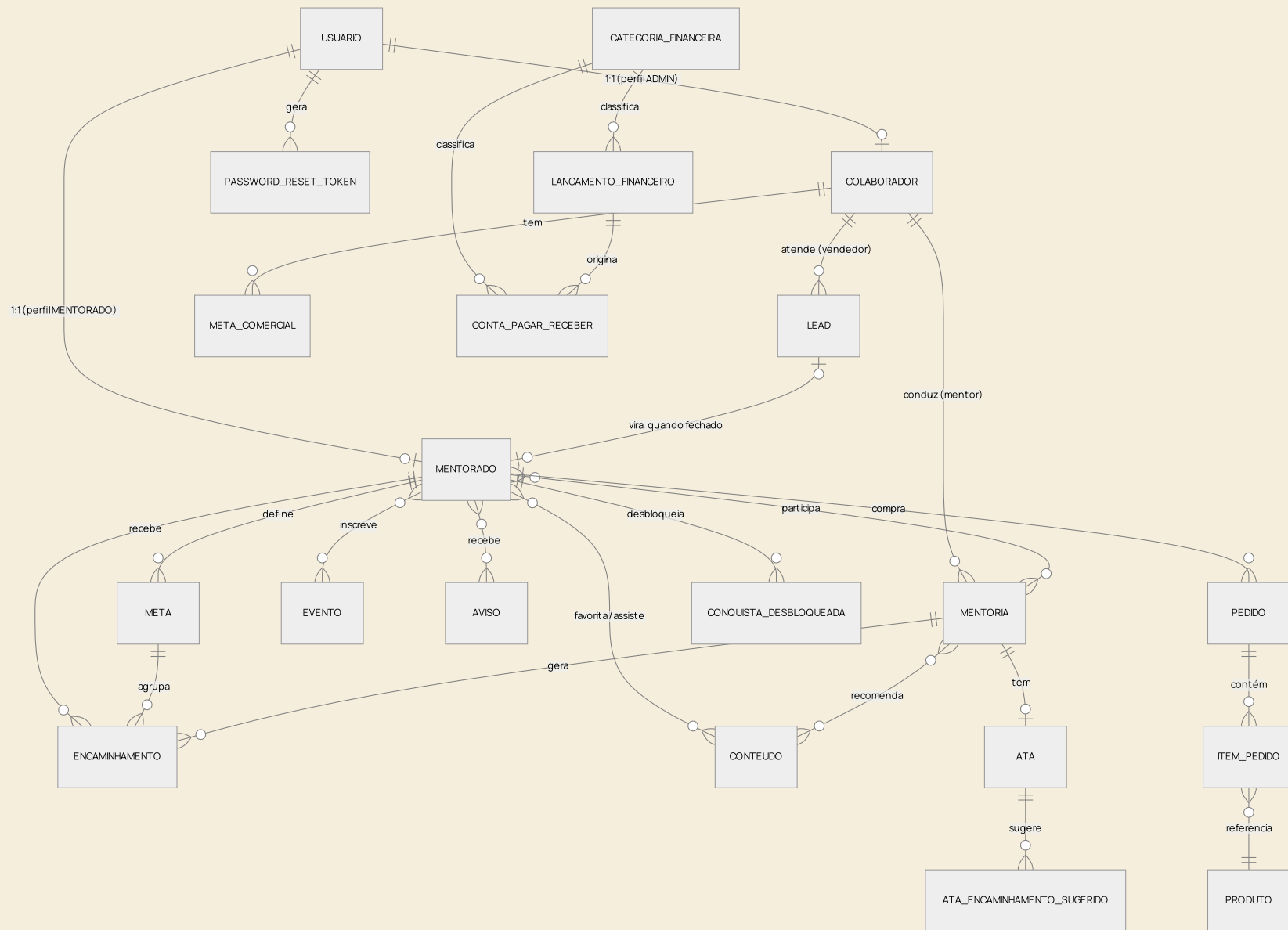
| DECISÃO                                | MOTIVO                                                                                                                                                     |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VPS única, não PaaS gerenciado         | Público 100% Brasil (CDN global não ajuda); front+back juntos elimina hop de rede; ~R\$ 35–55/mês vs ~R\$ 150–300/mês.                                     |
| Coolify                                | Cobre o que se perderia sem PaaS gerenciado: deploy via git push, build isolado do tráfego (sem downtime), TLS automático.                                 |
| Front e back atrás da mesma origem     | Sessão por cookie funciona sem CORS/SameSite=None — mesmo padrão do proxy do Vite em dev.                                                                  |
| Resolver dinâmico no Nginx do frontend | O IP do container do backend muda a cada deploy — sem <code>resolver 127.0.0.11 valid=10s</code> , o front fica respondendo 502 até ser reiniciado também. |

- **Trade-off aceito:** sem alta disponibilidade nativa — se a VPS cair, front e back caem juntos. Migração pra infra com HA (ex.: AWS) fica prevista para quando a escala justificar, não é requisito do MVP.

## 03 Modelo de dados

24 tabelas em `public`, todas com `id UUID` e a maioria com `versao BIGINT` (lock otimista — achado da revisão de segurança do E14: dupla liquidação concorrente podia duplicar lançamento no DRE). Colunas marcadas ● **cifrada** são `bytea` via `pgcrypto`, nunca usadas em `WHERE` / `ORDER BY` nativo do Postgres.

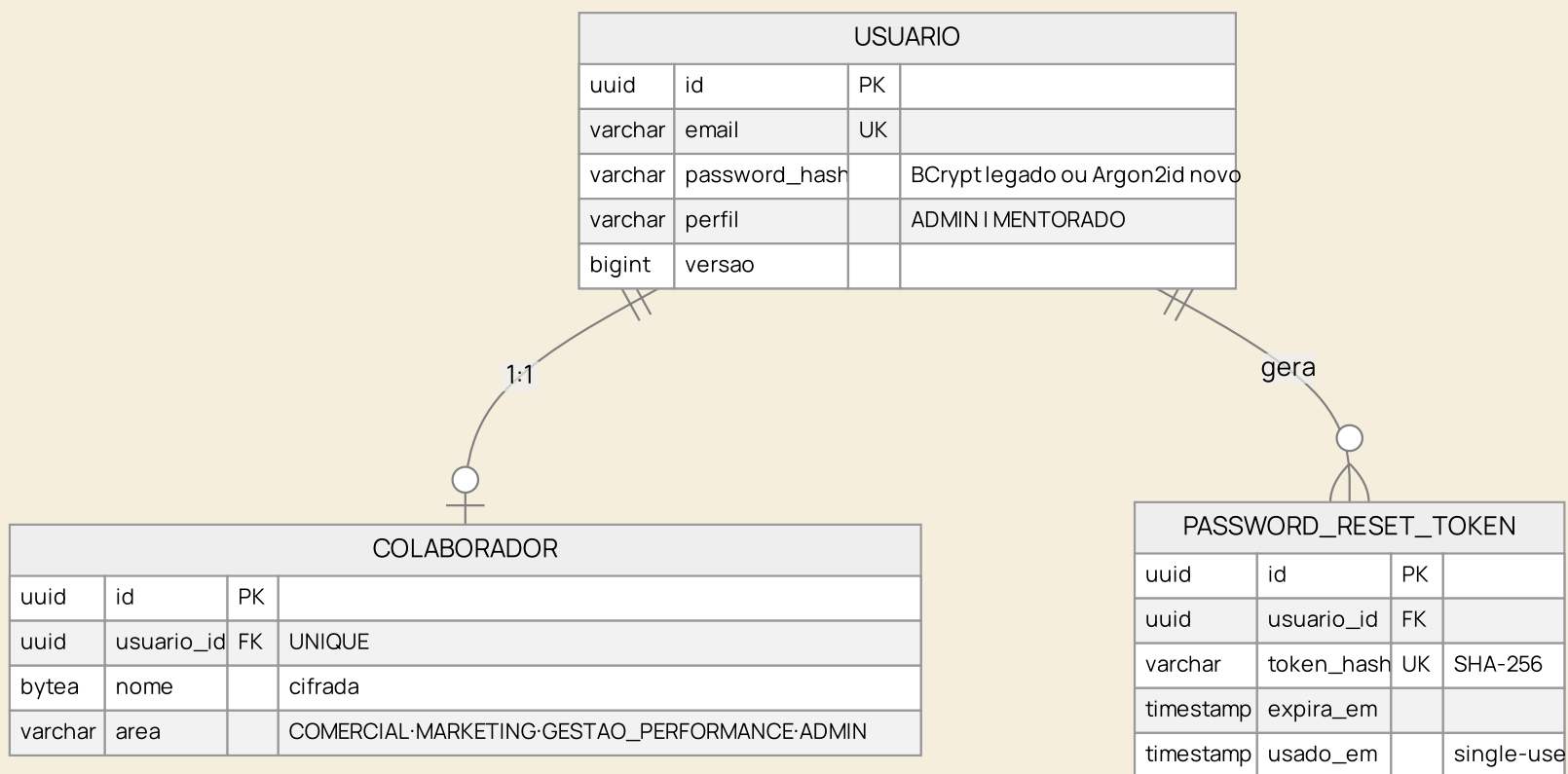
### Mapa geral





`atividade_log` não aparece aqui de propósito – é um log append-only sem FK (feed de "atividades recentes" do Dashboard Admin, popular via código em pontos de transição de status).

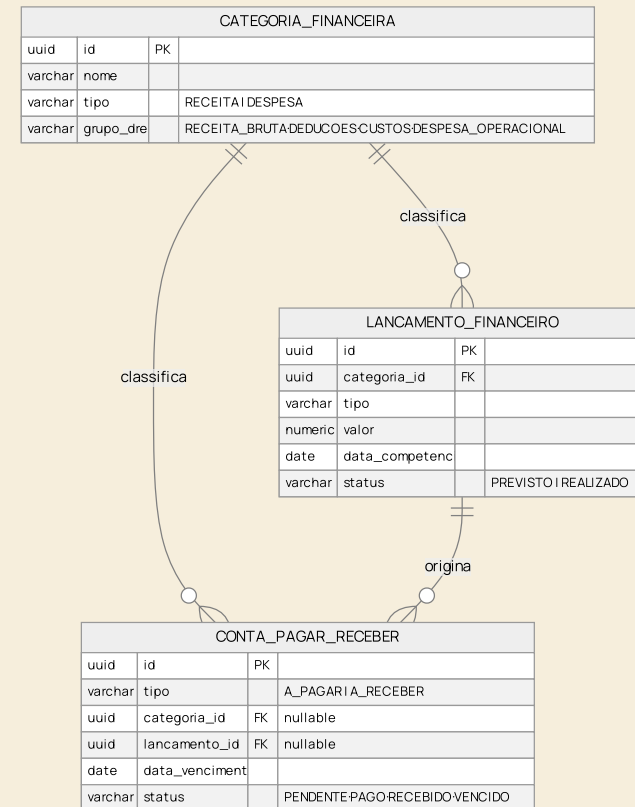
## Autenticação & Time (E1, E15)



## Comercial & Financeiro (E13, E14)

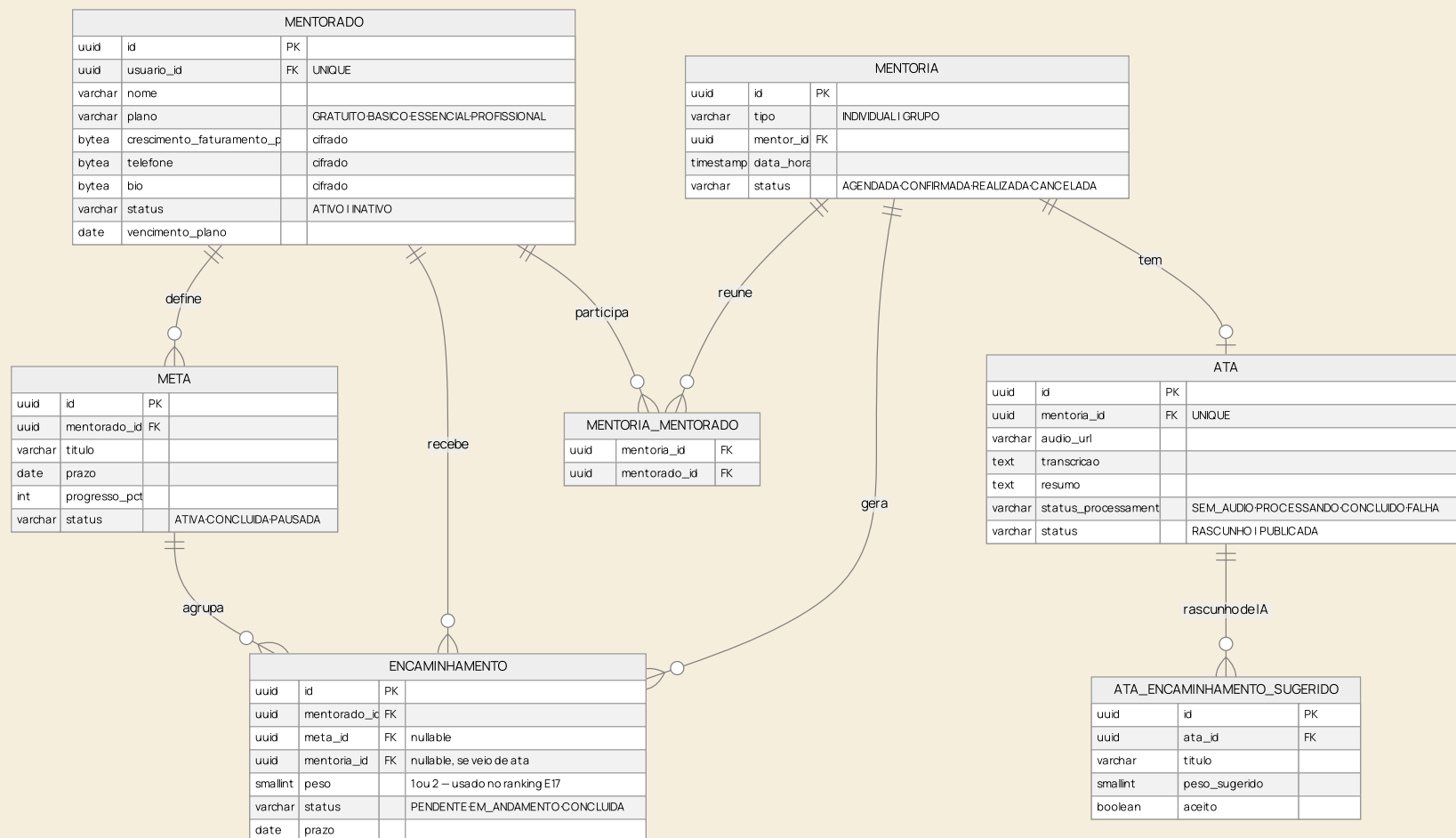
| LEAD    |              |    |                                                |
|---------|--------------|----|------------------------------------------------|
| uuid    | id           | PK |                                                |
| bytea   | nome         |    | cifrado                                        |
| bytea   | email        |    | cifrado                                        |
| bytea   | telefone     |    | cifrado                                        |
| varchar | status       |    | SOLICITACAOEM_CONTATO PROPOSTA FECHADO PERDIDO |
| uuid    | vendedor_id  | FK |                                                |
| uuid    | mentorado_id | FK | nullable, preenchido ao fechar                 |

| META_COMERCIAL |                 |    |  |
|----------------|-----------------|----|--|
| uuid           | id              | PK |  |
| uuid           | vendedor_id     | FK |  |
| int            | ano             |    |  |
| int            | mes             |    |  |
| int            | meta_fechamento |    |  |



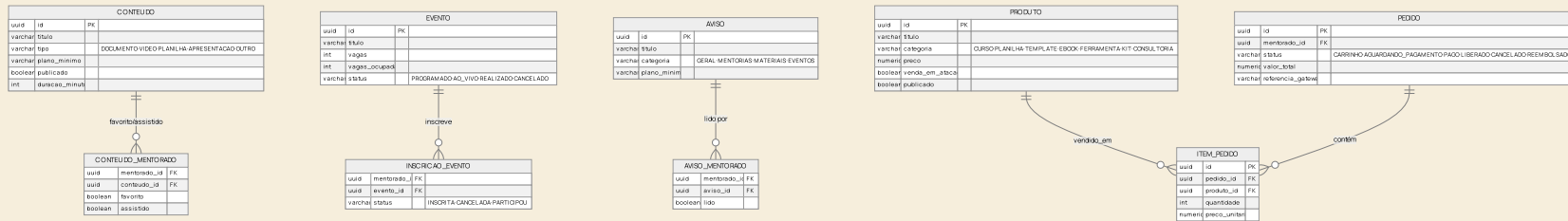
DRE e fluxo de caixa (`RelatorioFinanceiroService`) agregam `lancamento_financeiro` por `grupo_dre` / período – `valor` nunca é cifrado, é DRE interno da SAW, não "financeiro do mentorado" (a distinção que decide o que entra no pgcrypto, ver §06).

## Jornada do mentorado (E2–E5)



O rascunho da IA nunca vira **encaminhamento** real sozinho — só materializa quando a ata é publicada por um humano, um por um dos `ata_encaminhamento_sugerido.aceito = true`.

**Conteúdo, eventos, avisos e loja (E6–E8, E16)**



Só 1 `pedido` com status `CARRINHO` por mentorado — índice único parcial ( `WHERE status='CARRINHO'` ), não constraint de tabela inteira, já que o histórico pode ter vários pedidos `PAGO` / `LIBERADO` .

## 04 Módulos & épicos

Cada épico do `CLAUDE.md` mapeia pra um ou mais pacotes Java ( `com.sawhub.hub.*` ). A coluna "módulo RBAC" é o valor de `@RequiresModulo` que protege os endpoints Admin daquele pacote — telas do mentorado não usam módulo, só o perfil `MENTORADO` da sessão.

| ÉPICO | NOME                   | PACOTE             | MÓDULO RBAC       |
|-------|------------------------|--------------------|-------------------|
| E1    | Autenticação & Acesso  | security           | — (pré-requisito) |
| E2    | Dashboard do Mentorado | mentorado          | —                 |
| E3    | Metas Estratégicas     | meta               | —                 |
| E4    | Tarefas & Agenda       | mentorado (Tarefa) | —                 |
| E5    | Mentorias & Atas       | mentoria           | • MENTORADOS      |
| E6    | Materiais & Dicas      | conteudo           | • CONTEUDOS       |
| E7    | Eventos & Inscrições   | evento             | • CONTEUDOS       |
| E8    | Loja SAW               | loja               | • COMERCIAL       |
| E9    | Perfil & Gamificação   | perfil             | —                 |
| E10   | Painel Administrativo  | dashboardadmin     | • DASHBOARD       |
| E11   | Gestão Admin           | mentorado (Admin)  | • MENTORADOS      |
| E13   | Comercial & Vendas     | comercial          | • COMERCIAL       |
| E14   | Financeiro & DRE       | financeiro         | • FINANCEIRO      |
| E15   | Gestão de Time         | team               | • TIME            |
|       |                        |                    |                   |

| ÉPICO | NOME                         | PACOTE       | MÓDULO RBAC          |
|-------|------------------------------|--------------|----------------------|
| E16   | Avisos & Notificações        | aviso        | • CONTEUDOS          |
| E17   | Painel Consolidado & Ranking | consolidated | • PAINEL_CONSOLIDADO |

- Cada pacote costuma ter dois controllers quando o épico tem lado Admin e lado mentorado — ex.: `MentoriaController` (Admin, gate MENTORADOS) e `MentoriaMentoradoController` (mentorado, sem gate de módulo, só sessão).

## 05 Autenticação & RBAC

Dois perfis ( `Perfil` ): **MENTORADO** e **ADMIN**. Dentro do ADMIN, o acesso é refinado por **área** ( `Area` ) — a matriz Área × Módulo é lida de um único lugar ( `AreaModuloMatrix` ), tanto no login quanto em cada endpoint protegido, pra nunca duplicar a regra.

| ÁREA                  | MÓDULOS LIBERADOS                                        |
|-----------------------|----------------------------------------------------------|
| Comercial             | • COMERCIAL                                              |
| Marketing             | • CONTEUDOS                                              |
| Gestão de Performance | • MENTORADOS • CONTEUDOS • PAINEL_CONSOLIDADO            |
| Admin                 | • todos os 7 módulos (acesso irrestrito — cliente/sócio) |

## Fluxo de login (e-mail/senha)

- 1 **POST /api/v1/auth/login** — `JsonLoginFilter` intercepta antes do form-login padrão do Spring, lê e-mail/senha do corpo JSON.
- 2 `CustomUserDetailsService` resolve `Usuario` → verifica se é `Colaborador` ou `Mentorado`, monta as *authorities* ( `ROLE_ADMIN` / `ROLE_MENTORADO` + `MODULO_*` por área).
- 3 `DaoAuthenticationProvider` (padrão do Spring) chama `PasswordEncoder.matches()` — hash `Argon2id` pra contas novas, `BCrypt` legado continua validando via `setDefaultPasswordEncoderForMatches`.
- 4 Sessão persistida em **Redis** (Spring Session) — cookie `HttpOnly`, mesma origem front/back, sem necessidade de token JWT no client.
- 5 Toda requisição mutante exige **CSRF token** ( `CookieCsrfTokenRepository` ) — front lê o cookie e ecoa no header em cada POST/PUT/DELETE.

## Login com Google (OAuth2)

`ClientRegistration` montado manualmente (não o `CommonOAuth2Provider.GOOGLE` padrão) pra poder apontar as URLs via `GoogleOAuthProperties` — necessário pro ambiente E2E, que troca essas URLs por um stub local. Escopo `("email", "profile")`, sem `"openid"`, de propósito: evita o caminho OIDC do Spring, que exigiria JWKS real.

---

## 06 Segurança

Auth e Pagamento são os módulos de maior risco do projeto — passam por revisão de segurança dedicada antes de fechar. Camadas efetivamente implementadas e verificadas em produção:



### Em trânsito

TLS em toda comunicação (front ↔ back ↔ cliente), via Coolify/Let's Encrypt — renovação automática, sem intervenção manual.

### Senhas

Nunca em texto puro. `Argon2id` (OWASP) pra hashes novos, via `DelegatingPasswordEncoder` — `BCrypt` legado continua autenticando, migração transparente por troca de senha.

### Dados sensíveis em repouso

`pgcrypto` ( `pgp_sym_encrypt` ) nas colunas de PII/financeiro do mentorado que nunca entram em `WHERE` / `ORDER BY` — verificado ponta a ponta em produção (ver §03).

### Isolamento por tenant

Todo dado de mentorado é resolvido a partir do `principal` autenticado, nunca de um parâmetro de request — mentorado nunca acessa rota de outro mentorado.

### Lock otimista

`versao` em praticamente toda entidade — dois PUTs concorrentes na mesma linha falham limpo com 409, em vez de um sobrescrever o outro silenciosamente.

### Rate limiting

Filtros com contador no Redis pra endpoints públicos sensíveis a abuso: criação de lead e "esqueci minha senha" — ambos com janela e limite próprios.

- **Pendência conhecida:** criptografia de disco da VPS ainda não implementada (disco raiz `ext4` puro, sem LUKS) — não dá pra ativar a quente num servidor já em produção sem reprovisionar. Até lá, a proteção de dados em repouso depende só do `pgcrypto`.

- **Pendência conhecida:** o backup diário do Postgres existe e foi testado (restauração confirmada), mas hoje só fica *dentro* da própria VPS — a cópia off-site (fora do servidor) ainda não tem destino definido.

## 07 Integrações externas

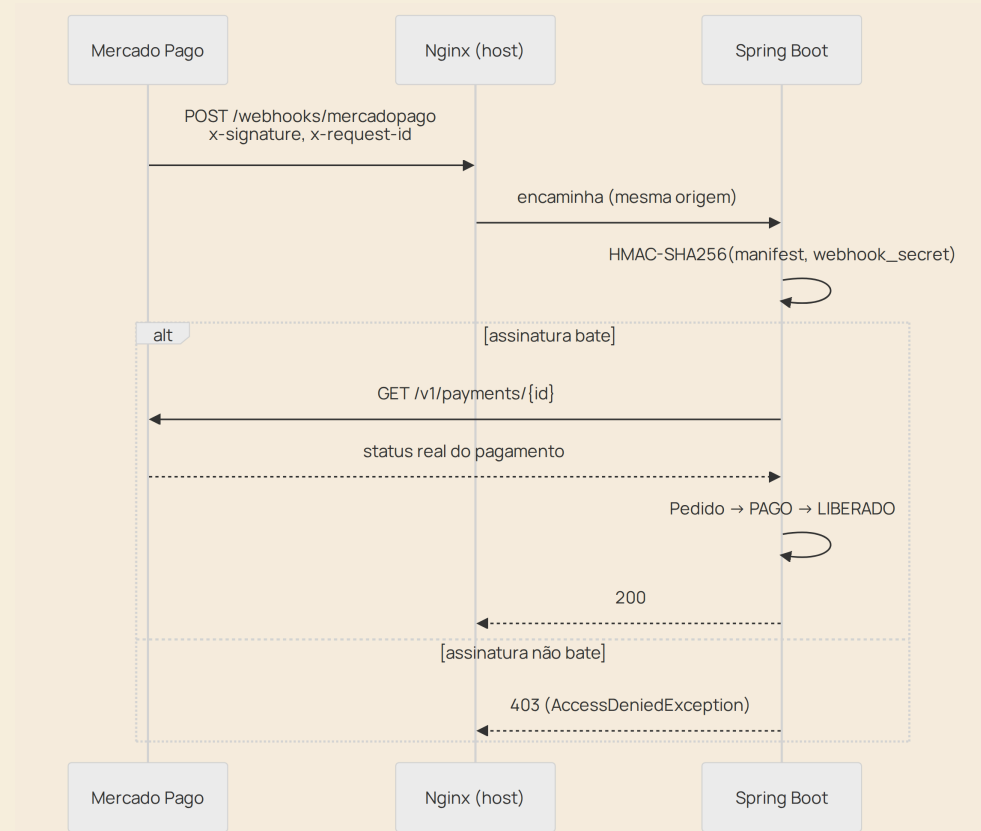
Toda integração externa segue o mesmo padrão de falha: sem credencial configurada, o recurso fica indisponível de forma limpa (exceção sentinela própria), nunca quebra o resto do sistema.

| SERVIÇO         | USO                                                                | PADRÃO DE CHAMADA                                                        |
|-----------------|--------------------------------------------------------------------|--------------------------------------------------------------------------|
| Google OAuth2   | Login social (E1/M07)                                              | Authorization Code flow, <code>ClientRegistration</code> manual          |
| Mercado Pago    | Checkout Pro — checkout, pagamento, webhook (E8)                   | <code>RestClient</code> direto (não SDK oficial), HMAC-SHA256 no webhook |
| Whisper         | Transcrição do áudio da mentoria (E5)                              | <code>RestClient</code> direto, timeout explícito                        |
| Claude Sonnet 5 | Rascunho de ata (resumo + encaminhamentos) a partir da transcrição | <code>RestClient</code> direto                                           |
| SMTP (Gmail)    | Recuperação de senha, notificações (E16)                           | <code>JavaMailSender</code> opcional — sem credencial, cai em log        |

- **Isolamento de segredo por ambiente:** as URLs de override desses serviços (pra apontar pros stubs do E2E) só existem em `application-e2e.yml`, carregado somente com `SPRING_PROFILES_ACTIVE=e2e`. O `application.yml` de produção não tem essa ponte de propósito — achado Alto da revisão de segurança: uma env mal configurada em produção redirecionando esses endpoints seria um vetor de account takeover ou vazamento de credencial real.

## Webhook do Mercado Pago

Endpoint público ( `/api/v1/webhooks/mercadopago` ), sem sessão — protegido só pela verificação de assinatura. Nunca confia no corpo da notificação: sempre reconsulta o pagamento de verdade na API do Mercado Pago a partir só do `id` recebido.



## 08 Deploy & operação

Três formas de subir o sistema, uma pra cada propósito — nunca misturar dev diário com a suíte E2E, os specs criam dados reais sem teardown.

| SCRIPT     | PROPÓSITO                                                                           | PORTAS      |
|------------|-------------------------------------------------------------------------------------|-------------|
| dev-up.sh  | Dev diário – backend/frontend nativos (hot-reload real), só infra em Docker         | 8080 / 5173 |
| e2e-up.sh  | Stack isolada só pro Playwright – banco/Redis próprios, reseed a cada subida        | 8090 / 5183 |
| full-up.sh | Sanity-check pré-deploy – stack inteira containerizada, mesma topologia da produção | 8080 / 8081 |

## Deploy em produção

- 1 **git push** pra `master` no repositório.
- 2 Coolify builda a imagem (Dockerfile próprio de `backend/` e `frontend/`) isolada do tráfego em produção – sem downtime.
- 3 Migrations do Flyway rodam automaticamente no boot do backend – sempre um arquivo `V{n}__descricao.sql` novo, nunca editar um já aplicado.
- 4 Health check confirma o container novo antes de trocar o tráfego.

## Backup

|             |                                                                |
|-------------|----------------------------------------------------------------|
| Script      | <code>/root/scripts/backup_saw_hub_postgres.sh</code> (na VPS) |
| Agendamento | Diário, 4:30 – cron do host                                    |
| Retenção    | 30 dias, limpeza automática                                    |
| Restauração | Testada – 28 tabelas, dados íntegros                           |

## 09 Como contribuir

Todo módulo novo segue o mesmo pipeline, sem exceção — inclusive fast-follows pequenos em áreas de risco alto (Auth, Pagamento).

- 1 **Blueprint** — desenhado e documentado antes de qualquer código: modelagem, contratos de API, rastreabilidade história ↔ módulo.
- 2 **TDD Red-Green-Refactor** — teste primeiro, de verdade, não teste-depois.
- 3 **Revisão de segurança** — obrigatória pra qualquer módulo de risco alto (Auth, Financeiro, Loja) antes de considerar fechado.
- 4 **E2E (Playwright)** — contra a stack real (Postgres + Redis + Spring + Vite), nunca mockada.
- 5 Todo bug achado em teste ad-hoc ou E2E vira **teste de regressão permanente**, nunca só um fix silencioso.

### Convenções de API

REST em `/api/v1`, JSON, erros padronizados. Datas em pt-BR, moeda em R\$.

### Migrations

Flyway, sempre incremental — uma migration já aplicada em qualquer ambiente nunca é editada, só uma `V{n+1}` nova.

- Ponteiros completos: `docs/spec.md` (histórias + BDD), `ROADMAP.md` (blueprint técnico por módulo), `docs/DEV.md` (ambiente local) e `CLAUDE.md` (visão de produto e princípios).

SAW HUB · Documentação técnica interna — gerada a partir do código e da infraestrutura em produção. Visual seguindo o padrão Onda (Maré Clara/Noite).